PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

Little Paws AI

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Piyush Kumar Sharma

2210990652

Pratham Paul Singh

2210990675

Supervised By:

Ashish Kumar

Mentor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"Little Paws AI"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Ashish Kumar The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **(Pratham Paul Singh)**

2210990675

Piyush Kumar Sharma

2210990652

Date :

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Ashish Kumar

Mentor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported and guided us throughout the development of the LilPaws AI project.

First and foremost, we would like to thank our mentors and faculty members at Chitkara University for their valuable guidance, continuous encouragement, and constructive feedback during the course of this project. Their insights played a crucial role in shaping both the technical and conceptual aspects of our work.

We are also grateful to our peers and team members for their collaboration, support, and exchange of ideas, which helped us overcome various challenges during development.

We would like to extend our appreciation to the open-source community and the developers behind technologies such as React.js, Node.js, MongoDB, FastAPI, and Redis, which made the development of this project possible.

Finally, we express our heartfelt thanks to our families for their constant support and motivation throughout this journey.

Abstract

LilPaws AI is an intelligent pet adoption platform designed to streamline and enhance the process of connecting rescued animals with suitable adopters. Traditional adoption systems often rely on manual screening, which can be time-consuming, inconsistent, and inefficient. This project addresses these challenges by integrating Artificial Intelligence to automate and optimize the adoption workflow.

The platform provides a comprehensive ecosystem that includes pet discovery, a multi-step digital adoption application, real-time status tracking, stray animal reporting, and an integrated e-commerce module for pet supplies. A key feature of the system is the AI-powered review service, which evaluates adoption applications using a Large Language Model to determine applicant suitability based on predefined criteria.

The system is built using a microservices-inspired architecture, consisting of a React-based frontend, a Node.js and Express backend, and a Python FastAPI-based AI service. Redis is used for queue-based asynchronous processing, while MongoDB serves as the primary database. This architecture ensures scalability, modularity, and efficient handling of concurrent requests.

The implementation demonstrates significant improvements in processing speed, decision consistency, and user experience. By automating initial screening and enabling real-time updates, the platform reduces the workload on shelter administrators while increasing transparency for users.

Overall, LilPaws AI showcases how the integration of AI with modern web technologies can create a smart, scalable, and efficient solution for real-world problems in pet adoption systems.

1. Introduction

LilPaws AI is a modern pet adoption platform designed to bridge the gap between rescued animals and potential adopters using intelligent automation. Traditional pet adoption systems often suffer from inefficiencies such as manual screening, delayed responses, and poor matching between pets and adopters. These issues can lead to unsuccessful adoptions and increased strain on animal shelters.

The LilPaws AI platform addresses these challenges by integrating Artificial Intelligence into the adoption workflow. It enables automated pre-screening of applicants, helping ensure that pets are placed in environments that best suit their needs. By leveraging AI-driven decision-making, the platform reduces the workload on shelter administrators while improving adoption success rates.

In addition to adoption management, the system provides a comprehensive ecosystem that includes pet discovery, application tracking, stray reporting, and an integrated e-commerce module for pet supplies. The platform is designed to support multiple shelters, specifically targeting regions such as Chandigarh, Panchkula, and Mohali, allowing centralized yet flexible management.

The architecture follows a microservices-inspired approach, separating concerns across frontend, backend, and AI services. This modular design improves scalability, maintainability, and performance. The frontend offers an interactive user experience, while the backend manages business logic and data flow. The AI service operates independently to process adoption applications asynchronously.

Overall, LilPaws AI aims to modernize the pet adoption process by making it faster, smarter, and more reliable, benefiting both shelters and adopters.

2. Methodology

The development of the LilPaws AI platform follows a structured and modular methodology, combining principles of full-stack development, microservices architecture, and AI-driven automation. The primary objective of this methodology is to ensure scalability, maintainability, and efficient processing of adoption workflows.

2.1 System Design Approach

The system is designed using a microservices-inspired architecture, where the application is divided into three major components:

- Frontend (Client Interface)
- Backend Server (Core Business Logic)
- AI Review Service (Intelligent Processing Unit)

Each component operates independently but communicates through well-defined APIs and message queues, ensuring loose coupling and high cohesion.

2.2 Data Flow and Processing

The workflow of the system is structured as follows:

1. **User Interaction**
   Users interact with the frontend to browse pets, submit adoption applications, or report stray animals.
2. **Request Handling**
   The frontend sends requests to the backend server, which validates and stores the data in the database.
3. **Queue-Based Processing**
   Adoption applications are pushed into a queue managed by Redis. This ensures that requests are handled asynchronously without blocking the main application.
4. **AI Evaluation**
   The AI Review Service retrieves queued applications and processes them using a Large Language Model (Google Gemini via LangChain). The AI evaluates applicant suitability based on predefined criteria.
5. **Result Storage and Notification**
   The AI-generated decision is stored back in the database. The backend updates the application status, which is reflected in real-time on the frontend dashboard.
6. **Admin Oversight**
   Shelter administrators can review AI decisions, override them if necessary, and finalize the adoption outcome.

2.3 AI Decision Framework

The AI system operates in two modes:

- **Autonomous Mode**: Automatically filters and approves/rejects applications.
- **Manual Oversight Mode**: Flags uncertain cases for human review.

The decision-making process considers factors such as applicant environment, pet compatibility, and adoption readiness.

2.4 Real-Time Updates

To enhance user experience, the system implements real-time updates using efficient state management and backend synchronization. This ensures that users and admins receive live status updates without manual refresh.

2.5 Security and Validation

The platform incorporates:

- JWT-based authentication for secure user sessions
- Input validation to prevent invalid or malicious data
- Role-based access control for admin and user separation

2.6 Development Strategy

An iterative development approach was followed:

- Initial setup of core features (authentication, pet listing)
- Integration of adoption workflow
- Implementation of AI-based screening
- Enhancement with additional modules (e-commerce, stray reporting)

This step-by-step approach ensured continuous testing and improvement of the system.

3. Tools and Technologies

The LilPaws AI platform is built using a combination of modern web development frameworks, backend technologies, and AI tools. Each technology is selected to ensure scalability, performance, and ease of development.

3.1 Frontend Technologies

The frontend is responsible for providing an interactive and user-friendly interface.

- **Framework: React.js (with Vite)**
  React.js is used to build a dynamic and component-based user interface. Vite is used as a fast build tool to improve development speed and performance.
- **Styling: Tailwind CSS & Shadcn UI**
  Tailwind CSS enables utility-first styling, allowing rapid UI development. Shadcn UI provides pre-built, customizable components for consistent design.
- **State Management: Redux Toolkit**
  Redux Toolkit is used to manage global application state such as user authentication, pet listings, and adoption status.
- **UI Enhancements: Lucide React & Framer Motion**
  Lucide React is used for icons, while Framer Motion adds smooth animations and transitions to improve user experience. 3.2 Backend Technologies

The backend handles business logic, API management, and data processing.

- **Runtime Environment: Node.js**
  Node.js enables scalable server-side execution using JavaScript.
- **Framework: Express.js**
  Express.js is used to build RESTful APIs and handle routing efficiently.
- **Database: MongoDB (with Mongoose ODM)**
  MongoDB stores application data such as users, pets, and adoption records. Mongoose provides schema-based data modeling.
- **Caching & Queue System: Redis (ioredis)**
  Redis is used for caching and managing asynchronous job queues for adoption applications.
- **Cloud Storage: Cloudinary**
  Cloudinary handles image uploads and storage for pet profiles.
- **Payment Integration: PayPal SDK**
  PayPal is used to process secure online payments in the e-commerce module. 3.3 AI Review Service Technologies

This component is dedicated to intelligent processing of adoption applications.

- **Framework: FastAPI (Python)**
  FastAPI is used to build a high-performance API for AI processing.

- **LLM Integration: LangChain + Google Gemini Pro**
  LangChain is used to structure prompts and workflows, while Google Gemini Pro acts as the core language model for evaluating adoption applications.
- **Asynchronous Processing: Background Workers**
  The AI service processes applications asynchronously to ensure non-blocking performance.
- **Database Access: Motor (Async MongoDB Driver)**
  Motor enables asynchronous interaction with MongoDB for faster data operations. 3.4 DevOps and Environment Tools

- **Version Control: Git & GitHub**
  Used for source code management and collaboration.
- **Environment Management: .env Configuration**
  Sensitive data such as API keys and database URIs are managed securely using environment variables.
- **Local Development Tools**
  Node.js, Python, Redis Server, and MongoDB are used for local development and testing.

3.5 Architectural Highlights

- Microservices-inspired structure improves scalability
- Asynchronous processing enhances performance
- Separation of concerns ensures maintainability
- AI integration enables intelligent decision-making

4. Implementation

The implementation of LilPaws AI focuses on integrating multiple system components—frontend, backend, and AI services—into a cohesive and functional platform. The development follows a modular approach, ensuring that each component can be developed, tested, and deployed independently.

4.1 Frontend Implementation

The frontend is developed using React.js with Vite, following a component-based architecture.

- **Component Structure**
  The UI is divided into reusable components such as pet cards, forms, dashboards, and navigation bars. This improves maintainability and scalability.
- **Routing and Navigation**
  Client-side routing is implemented to enable seamless navigation between pages such as pet listings, adoption forms, and admin dashboards.
- **State Management**
  Redux Toolkit is used to manage global state, including user authentication, pet data, and application status.
- **Form Handling**
  Multi-step adoption forms are implemented with validation to ensure accurate data collection from users.
- **Real-Time Updates**
  The frontend dynamically updates application statuses by fetching updated data from the backend, providing a smooth user experience.

4.2 Backend Implementation

The backend is built using Node.js and Express.js, serving as the core of the system.

- **API Development**
  RESTful APIs are created to handle operations such as user authentication, pet management, adoption applications, and order processing.
- **Authentication and Authorization**
  JWT-based authentication is implemented to secure user sessions. Role-based access control ensures that only authorized admins can access certain features.
- **Database Integration**
  MongoDB is used to store structured data. Mongoose schemas define collections such as Users, Pets, Applications, and Orders.
- **File Handling**
  Cloudinary integration allows efficient uploading and retrieval of pet images.
- **Queue Management**
  Redis is used to implement a queue system where adoption applications are stored for asynchronous AI processing.

4.3 AI Review Service Implementation

The AI Review Service is implemented as a separate microservice using FastAPI.

- **API Endpoints**
  Endpoints are created to receive adoption applications from the queue and process them.
- **AI Processing Pipeline**
  Applications are evaluated using LangChain, which structures prompts sent to the Google Gemini model. The AI generates decisions such as approval, rejection, or flagging for review.
- **Asynchronous Workers**
  Background workers continuously monitor the Redis queue and process applications without blocking other services.
- **Result Integration**
  Processed results are stored back in MongoDB and made available to the backend for further action.

4.4 Integration of Components

- The frontend communicates with the backend via REST APIs.
- The backend pushes adoption requests to Redis queues.
- The AI service consumes these requests, processes them, and updates the database.
- The frontend reflects updated statuses in real time.

This pipeline ensures efficient communication and smooth operation across all system components.

4.5 Deployment and Execution

The system is executed in three main steps:

1. **Backend Server**
   Installed dependencies and started using development scripts.
2. **AI Service**
   Python environment setup and execution of FastAPI server.
3. **Frontend Application**
   Built and served using Vite development server.

Additionally, initial data such as shelters and pets can be seeded into the database using predefined scripts.

4.6 Error Handling and Optimization

- Proper error handling mechanisms are implemented in APIs.
- Input validation ensures data integrity.
- Asynchronous processing reduces system load.

- Efficient state updates improve frontend performance.

Overall, the implementation ensures that LilPaws AI operates as a scalable, efficient, and intelligent platform capable of handling real-world adoption workflows.

5. Major Findings

The development and implementation of the LilPaws AI platform resulted in several significant outcomes, both in terms of technical performance and real-world usability. The integration of AI with a full-stack system demonstrated clear improvements over traditional pet adoption processes.

5.1Improved Adoption Screening Efficiency

One of the key outcomes is the automation of the adoption screening process.AI-based evaluation significantly reduces manual workload for shelter administrators. Applications are processed faster compared to traditional manual review systems. The system can handle multiple applications simultaneously without delays.

5.2 Enhanced Decision-Making with AI

The integration of an AI review system provides more structured and consistent decision-making. Applications are evaluated based on predefined criteria using a Large Language Model. The system ensures unbiased initial screening. Edge cases are flagged for human review, maintaining reliability.

5.3 Real-Time Application Tracking

The platform successfully implements live status tracking.Users can monitor their application progress (AI Review → Final Decision). Admins receive real-time updates on pending applications. This improves transparency and user engagement.

5.4 Scalable and Modular Architecture

The micro services-inspired architecture proved effective.Independent services (frontend, backend, AI) improve scalability. Asynchronous processing using Redis prevents system bottlenecks. The system can be extended easily with additional features or services.

5.5 Improved User Experience

The platform provides a seamless and intuitive interface.Fast navigation and responsive UI enhance usability. Multi-step forms guide users effectively through the adoption process. Integrated features (pet discovery, e-commerce, stray reporting) create a complete ecosystem.

5.6 Multi-Shelter Management Capability

The system successfully supports multiple shelters.Admin dashboards allow independent control for different locations. Centralized data management ensures consistency. This makes the platform suitable for real-world deployment across regions.

5.7 Reliable Data Handling and Security

Secure authentication using JWT ensures safe access. Data validation reduces errors and inconsistencies. Cloud storage integration ensures efficient media handling.

5.8 Performance Optimization

Queue-based processing improves response time. Asynchronous AI evaluation prevents blocking of core services. Efficient state management enhances frontend performance.

5.9 Practical Impact

Reduces time required for pet adoption decisions. Increases chances of successful pet-owner matching. Encourages community participation through stray reporting.

6. Conclusion and Future Scope

6.1 Conclusion

LilPaws AI successfully demonstrates how modern web technologies combined with Artificial Intelligence can transform the traditional pet adoption process into a more efficient, scalable, and intelligent system. The platform addresses key challenges such as manual screening delays, inconsistent decision-making, and lack of transparency by introducing AI-assisted evaluation and real-time tracking.

The microservices-inspired architecture ensures separation of concerns, allowing the frontend, backend, and AI services to function independently while maintaining smooth communication. The use of asynchronous processing further enhances system performance by handling multiple adoption requests without blocking operations.

From a user perspective, the platform provides an intuitive interface for discovering pets, submitting applications, and tracking progress. For administrators, it offers powerful tools such as AI-assisted screening, real-time dashboards, and multi-shelter management. Overall, the system improves both operational efficiency and user experience.

6.2 Future Scope

While LilPaws AI is a fully functional platform, there are several areas where it can be further enhanced:

- **Advanced AI Models**
  Improve the accuracy of adoption decisions by incorporating more advanced models, better training data, and feedback loops from previous decisions.
- **Mobile Application Development**
  Develop dedicated Android/iOS applications to increase accessibility and user engagement.
- **Real-Time Notifications**
  Integrate push notifications, email alerts, or SMS updates for application status and important events.
- **Geolocation-Based Features**
  Enable location-based pet discovery and stray reporting using maps and GPS integration.
- **Integration with Veterinary Services**
  Partner with veterinary clinics to provide health records, vaccination tracking, and post-adoption care.
- **Enhanced E-commerce Module**
  Expand the marketplace with more vendors, subscription-based pet care services, and personalized product recommendations.
- **Analytics and Reporting Dashboard**
  Provide detailed insights for shelters, such as adoption rates, user engagement, and AI decision accuracy.

- **Security Enhancements**
  Implement advanced security features such as multi-factor authentication and improved data encryption.
- **Scalability for Nationwide Deployment**
  Extend support beyond specific regions to enable nationwide or global adoption networks.

References

1. React.js Documentation. Available at: https://react.dev/
2. Node.js Documentation. Available at: https://nodejs.org/
3. Express.js Documentation. Available at: https://expressjs.com/
4. MongoDB Documentation. Available at: https://www.mongodb.com/docs/
5. Redis Documentation. Available at: https://redis.io/documentation
6. FastAPI Documentation. Available at: https://fastapi.tiangolo.com/
7. LangChain Documentation. Available at: https://docs.langchain.com/
8. Google Gemini AI Documentation. Available at: https://ai.google.dev/
9. Cloudinary Documentation. Available at: https://cloudinary.com/documentation
10. PayPal Developer Documentation. Available at: https://developer.paypal.com/docs/
11. Tailwind CSS Documentation. Available at: https://tailwindcss.com/docs
12. Redux Toolkit Documentation. Available at: https://redux-toolkit.js.org/
13. Framer Motion Documentation. Available at: https://www.framer.com/motion/
14. Lucide Icons Documentation. Available at: https://lucide.dev/

Appendices

The appendices section provides supplementary information that supports the main content of the report. It includes technical configurations, setup instructions, and additional implementation details of the LilPaws AI platform.

Appendix A: System Requirements

Hardware Requirements

- Minimum 8 GB RAM
- Dual-core processor or higher
- Stable internet connection

Software Requirements

- Node.js (v18 or above)
- Python (v3.9 or above)
- MongoDB (Local or Atlas)
- Redis Server
- Code Editor (e.g., VS Code)

Appendix B: Environment Configuration

Backend (.env)

```
MONGODB_URL=your_mongodb_uri
REDIS_URL=redis://localhost:6379
JWT_SECRET=your_secret
CLOUDINARY_CLOUD_NAME=your_name
CLOUDINARY_API_KEY=your_key
CLOUDINARY_API_SECRET=your_secret
```

AI Service (.env)

```
MONGODB_URL=your_mongodb_uri
REDIS_URL=redis://localhost:6379
GEMINI_API_KEY=your_gemini_key
```

Appendix C: Installation and Setup

Step 1: Backend Setup

```
cd server
npm install
npm run dev
```

Step 2: AI Service Setup

```
cd ai-review-service
pip install -r requirements.txt
python main.py
```

Step 3: Frontend Setup

```
cd client
npm install
npm run dev
```

Appendix D: Database Seeding

To populate initial data (shelters and pets):

```
node seed Shelters.js
```

Admin registration path:

```
/auth/register-shelter-admin
```

Appendix E: Project Structure Overview

```
/client          → Frontend (React)
/server          → Backend (Node.js + Express)
/ai-review-service  → AI Service (FastAPI)
```

This appendix section provides all necessary technical details required to understand, run, and evaluate the LilPaws AI system.